

Database Documentation (A–Z)

DATABASE DOCUMENTATION (A–Z)

Project: Government Schemes Eligibility & Recommendation Platform

Database Name: scheme_platform

DBMS: MySQL 8.0

Final Table: schemes

Final Rows: 20

1) Database Selection & Initial Verification

1.1 List all databases

SHOW DATABASES;

Purpose: Confirms MySQL is running and databases exist.

1.2 Select project database

USE scheme_platform;

Purpose: Ensures all operations run inside correct database.

1.3 List tables in the database

SHOW TABLES;

Purpose: Confirms existing tables before cleanup or migration.

2) Importing Dataset into MySQL (Schemes Table)

2.1 Import approach used

- Dataset prepared as CSV (later fixed due to formatting/mapping issues).

- Imported using MySQL Workbench / CSV import wizard.

- Table name created: schemes_dataset_v2 (temporary).

Issue: Import created wrong datatypes (TEXT columns).

Fix: Converted to professional schema later.

3) Database Cleanup (Keep Only Dataset Table)

3.1 Goal

Keep only the dataset table and remove other tables to avoid FK errors and confusion.

3.2 Dropping unwanted tables safely (FK-safe order)

DROP TABLE IF EXISTS eligibility_results;

DROP TABLE IF EXISTS eligibility_request;

DROP TABLE IF EXISTS user_profile;

DROP TABLE IF EXISTS users;

DROP TABLE IF EXISTS schemes;

Purpose: Removes old tables and foreign key dependencies.

3.3 Rename dataset table to final name

RENAME TABLE schemes_dataset_v2 TO schemes;

Purpose: Final table name standardized to schemes.

3.4 Confirm final tables

SHOW TABLES;

Expected: only schemes

4) Initial Data Verification

4.1 Confirm total records

SELECT COUNT(*) FROM schemes;

Expected: 20

5) Problem: Wrong Datatypes After Import (TEXT/BLOB Index Error)

5.1 Attempt to add indexes (failed)

ERROR 1170: BLOB/TEXT column used in key specification without a key length

Cause: state/category/min_age/income_max imported as TEXT.

5.2 Check current schema

SHOW CREATE TABLE schemes;

Observed issues:

- state TEXT
- category TEXT
- min_age TEXT
- income_max TEXT
- scheme_id not PRIMARY KEY

6) Professional Schema Migration (Final Fix)

6.1 Create a clean professional table

```
CREATE TABLE schemes_new (  
  scheme_id INT NOT NULL,  
  scheme_name VARCHAR(255) NOT NULL,  
  state VARCHAR(50) NOT NULL,  
  category VARCHAR(100) NOT NULL,  
  eligibility TEXT NOT NULL,  
  benefits TEXT NOT NULL,  
  documents TEXT NULL,  
  apply_link TEXT NULL,  
  min_age INT NULL,  
  max_age INT NULL,  
  income_max INT NULL,  
  gender_allowed ENUM('Any','Male','Female','Other') NOT NULL DEFAULT 'Any',  
  rules_verified BOOLEAN NOT NULL DEFAULT FALSE,  
  rule_source VARCHAR(255) NULL,  
  PRIMARY KEY (scheme_id),  
  INDEX idx_schemes_state (state),  
  INDEX idx_schemes_category (category),  
  INDEX idx_schemes_verified (rules_verified),  
  INDEX idx_schemes_gender (gender_allowed),  
  INDEX idx_schemes_age (min_age, max_age),  
  INDEX idx_schemes_income (income_max)  
);
```

7) Data Migration from Old Table to New Table

7.1 Copy + convert datatypes safely

```
INSERT INTO schemes_new (  
  scheme_id, scheme_name, state, category, eligibility, benefits, documents, apply_link,  
  min_age, max_age, income_max, gender_allowed, rules_verified, rule_source  
)  
SELECT  
  scheme_id,  
  LEFT(scheme_name,255),  
  LEFT(state,50),  
  LEFT(category,100),  
  eligibility,  
  benefits,  
  documents,  
  apply_link,  
  NULLIF(min_age,"") + 0,  
  NULLIF(max_age,"") + 0,  
  NULLIF(income_max,"") + 0,  
  CASE  
  WHEN gender_allowed IN ('Any','Male','Female','Other') THEN gender_allowed  
  ELSE 'Any'
```

```
END,  
IFNULL(rules_verified,0),  
LEFT(rule_source,255)  
FROM schemes;
```

8) Replace Old Table With Final Table

```
DROP TABLE schemes;  
RENAME TABLE schemes_new TO schemes;
```

9) Final Schema Validation

```
DESC schemes;  
SELECT COUNT(*) FROM schemes;  
SHOW INDEX FROM schemes;
```

10) Eligibility Filtering Queries (DB-Level Matching)

10.1 Verified rules only

```
SELECT s.scheme_id, s.scheme_name, s.state, s.category,  
s.min_age, s.max_age, s.income_max, s.gender_allowed, s.rules_verified  
FROM schemes s  
WHERE s.state = 'Telangana'  
AND (s.rules_verified = 1)  
ORDER BY s.scheme_id;
```

10.2 User-like eligibility filtering example

```
SELECT s.scheme_id, s.scheme_name, s.category, s.apply_link  
FROM schemes s  
WHERE s.state = 'Telangana'  
AND (s.min_age IS NULL OR 24 >= s.min_age)  
AND (s.max_age IS NULL OR 24 <= s.max_age)  
AND (s.income_max IS NULL OR 150000 <= s.income_max)  
AND (s.gender_allowed = 'Any' OR s.gender_allowed = 'Male')  
ORDER BY s.rules_verified DESC, s.scheme_id;
```

11) Strong Data Integrity (University-grade)

11.1 Restrict state values

```
ALTER TABLE schemes  
MODIFY state ENUM('Central','Telangana') NOT NULL;
```

11.2 Add CHECK constraints

```
ALTER TABLE schemes  
ADD CONSTRAINT chk_min_age CHECK (min_age IS NULL OR min_age BETWEEN 0 AND 120),  
ADD CONSTRAINT chk_max_age CHECK (max_age IS NULL OR max_age BETWEEN 0 AND 120),  
ADD CONSTRAINT chk_age_range CHECK (  
(min_age IS NULL OR max_age IS NULL) OR (min_age <= max_age)  
),  
ADD CONSTRAINT chk_income_max CHECK (income_max IS NULL OR income_max >= 0);
```

12) Database Views

12.1 View for verified schemes

```
CREATE OR REPLACE VIEW v_verified_schemes AS  
SELECT scheme_id, scheme_name, state, category, min_age, max_age, income_max, gender_allowed,  
rule_source  
FROM schemes  
WHERE rules_verified = 1;
```

12.2 View for Telangana schemes

```
CREATE OR REPLACE VIEW v_telangana_schemes AS  
SELECT scheme_id, scheme_name, category, apply_link
```

```
FROM schemes
WHERE state = 'Telangana';
```

13) Encoding Cleanup (Rule Source Text Fix)

```
UPDATE schemes SET rule_source = REPLACE(rule_source, 'â€‘', '-');
UPDATE schemes SET rule_source = REPLACE(rule_source, 'â‰¤', '<=');
UPDATE schemes SET rule_source = REPLACE(rule_source, 'â‚¹', 'Rs');
```

14) Final Audit Queries (For Report Screenshots)

```
SHOW CREATE TABLE schemes;
SHOW INDEX FROM schemes;
SELECT * FROM v_verified_schemes;
SELECT * FROM v_telangana_schemes;
SELECT rules_verified, COUNT(*) AS cnt FROM schemes GROUP BY rules_verified;
SELECT state, COUNT(*) AS cnt FROM schemes GROUP BY state;
```

FINAL DATABASE SUMMARY

Objects:

- Table: schemes
- Views: v_verified_schemes, v_telangana_schemes
- Indexes: state, category, verified flag, gender, age range, income
- Constraints: age validity, age range, income non-negative

Row count: 20

Outcome: Database is structured, validated, scalable, and ready for backend API integration.